

REST API CLIENT

SPIS TREŚCI

Spis treści.....	1
Cel zajęć.....	1
Rozpoczęcie.....	1
Uwaga.....	1
Wymagania.....	2
Badanie API.....	2
Implementacja.....	2
Commit projektu do GIT.....	4
Podsumowanie.....	4

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- pobieranie danych z zewnętrznych zasobów za pomocą REST API
- zdobywanie wiedzy na temat zewnętrznych API za pomocą dokumentacji typu Swagger
- wysyłanie asynchronicznych żądań z wykorzystaniem XMLHttpRequest i Fetch API

W praktycznym wymiarze uczestnicy stworzą dynamiczną stronę HTML pozwalającą na wyświetlanie bieżącej informacji pogodowej oraz prognoz dla zadanej przez użytkownika miejscowości.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie wykonywania połączeń synchronicznych i asynchronicznych z poziomu JS na stronie.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w poprzednio, wejdź do właściwości dokumentu i zaktualizuj niestandardowe pola.

WYMAGANIA

Szablon plików wymaganych w LAB D został dostarczony razem z instrukcją.

W ramach LAB D przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS i stylami z zewnętrznego pliku CSS;
- pole tekstowe (input typu „text”) do wprowadzania adresu
- przycisk „Pogoda”, po kliknięciu którego wykonywane jest zapytanie asynchroniczne:
 - do API Current Weather: <https://openweathermap.org/current> za pomocą XMLHttpRequest
 - do API 5 day forecast: <https://openweathermap.org/forecast5> za pomocą Fetch API
- obsługa zwrotki z obu API – wypisanie pogody bieżącej oraz prognoz poniżej pola wyszukiwania.
- wymagane jest użycie własnego klucza do API, w przypadku użycia cudzego klucza praca nie zostanie oceniona
- aplikacja musi być responsywna oraz różnić się wyglądem od zaprezentowanego poniżej, w przypadku wykorzystania poniższego wyglądu praca nie zostanie oceniona



Wygeneruj klucz do API. Ponieważ aktywacja może chwilę potrwać, na czas trwania laboratorium możesz wykorzystać „służbowy” klucz: `7ded80d91f2b280ec979100cc8bbba94`. **UWAGA!** Klucz zostanie dezaktywowany niedługo po zajęciach. Musisz wygenerować swój własny.

Prowadzący omówi powyższe wymagania. Upewnij się, czy wszystko rozumiesz.

W przypadku blokady twórczej można posilkować się filmem: <https://www.youtube.com/watch?v=WoKp2qDFxKk> jednakże spróbuj rozwiązać ten problem samodzielnie! Wygląd aplikacji musi różnić się od prezentowanego na filmie!

Tu umieść swoje notatki:

...notatki...

BADANIE API

Poświęć kilka minut na wykonanie przykładowych zapytań do API z poziomu pasku adresu przeglądarki. Podaj wymagane parametry dla osiągnięcia różnych wyników. Zbadaj odpowiedzi API, aby uzyskać pełen obraz wymagań i możliwości API.

Wstaw zrzut ekranu zawierający Twój własny klucz na stronie <https://www.openweathermap.org> (zakładka API keys)

9f117a3429baad80f475d096a5e1e8fc

hej1

Active



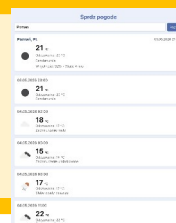
oraz fragment kodu, w którym wykorzystany zostaje Twój własny klucz

```
const API_KEY : string = '9f117a3429baad80f475d096a5e1e8fc';
```

IMPLEMENTACJA

Tradycyjnie implementację należy zacząć od zbudowania w HTML + CSS wszystkich wymaganych elementów / placeholderów na te elementy. Następnie krok po kroku należy implementować poszczególne zachowania.

Wstaw zrzut ekranu zawierającego stronę ze wszystkimi elementami, tj. pole tekstowe, przycisk, miejsce do wyświetlenia pogody i prognozy:

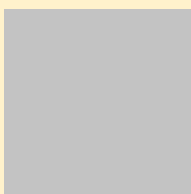


Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do current za pomocą XMLHttpRequest:

```
function getCurrentWeatherXHR(city) : Promise<unknown> {
  return new Promise( executor: (resolve, reject) : void => {
    const xhr : XMLHttpRequest = new XMLHttpRequest();
    const url : string = `${API_BASE}/weather?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric&lang=pl`;
    xhr.open( method: 'GET', url);
    xhr.responseType = 'json';
    xhr.onload = () : void => {
      if (xhr.status === 200) {
        resolve(xhr.response);
      }
    };
    xhr.onerror = () => reject( reason: { message: 'błąd xhr' } );
    xhr.send();
  });
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

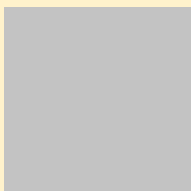


Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do forecast za pomocą Fetch:

```
function getForecastFetch(city) : Promise<any> {
  const url : string = `${API_BASE}/forecast?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric&lang=pl`;
  return fetch(url).then(resp : Response => {
    if (!resp.ok) {
      return resp.json().then(j => { throw j; }).catch(() : never => { throw { message: `${resp.status} ${resp.statusText}` }; });
    }
    return resp.json();
  }).catch(err => {
    err = null;
    return null;
  });
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.



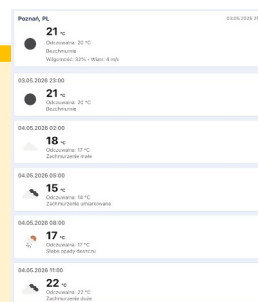
Punkty:

0

1

Wstaw zrzut ekranu przedstawiający wizualizację prognoz pogody:

Upewnij się, że widoczne są pasek wyszukiwania ze wskazaną miejscowością, a także zarówno pogoda bieżąca jak i prognozy pogody.



Punkty:

0

1

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Upewnij się, że aktualna wersja rozwiązania jest dostępna przez GitHub Pages.

Podaj link do swojego repozytorium GitHub Pages:

...link, np. <https://github.com/.../...> ...

<https://github.com/barczykjakub553-ui/barczykjakub553-ui.github.io/tree/main/lab-d>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

Okej ale ile można robić css i html nonstop. Fajnie pokazane używanie api call i ich odbieranie. Bardzo mi się podoba bo akurat takie rzeczy wykorzystuje w swojej aplikacji (nie dokładnie pogoda ale ogólnie API).

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.